

GapHarness: Obligation-First Minimal Harness Synthesis for API-Only LLM Agents

Anonymous Authors

Abstract

API-only LLM agents need external runtime support for retrieval, execution, state, action, control, and verification. Tool routers often choose tools directly, conflating what a query requires with which runtime modules happen to exist. GapHarness separates these questions: it first infers external obligations, then compiles the lowest-cost declared runtime harness that covers them. On the 1000-task human-audited GapBench controlled benchmark, GapHarness matches the oracle minimal harness under gold obligations. With LLM-inferred obligations, it outperforms direct and router baselines while exposing calibration tradeoffs. Registry guarding reduces a systematic sandbox-action unsupported false-positive failure. Anti-circularity stress tests show that success depends on declared registry affordances and meaningful obligation labels.

1. Motivation

Tool-using LLM agents and reasoning-action systems show that external actions can improve model behavior (Yao et al. 2023; Schick et al. 2023; Patil et al. 2023; Qin et al. 2024). But tool choice is only one layer. A query may require observation beyond the prompt, deterministic execution, durable state, sandbox action, risk control, or independent verification. These are obligations, not tool names.

GapHarness asks a narrow systems question:

Given a user query, an obligation profile, and a declared registry of modules, can we compile the minimal harness that satisfies the external obligations required for a warranted answer or action?

This is not another general agent framework. It is a compiler-style layer between obligation inference and runtime module selection. Minimality is relative to a declared ontology, registry, dependency model, and cost function.

2. Method

GapHarness uses six obligations: Observation, Execution, State, Action, Control, and Verification. A profiler emits obligations, required capabilities, output contracts, and unsupported or clarification conditions. A registry declares modules, affordances, dependencies, and cost. The compiler performs exact search over registry subsets and returns the lowest-cost subset that covers the profile. If no subset covers the profile, it returns unsupported or clarify rather than hallucinating support.

3. Benchmark and Metrics

GapBench-1000 is a controlled factorial benchmark with human-audited labels for obligations, required capabilities, oracle minimal harnesses, expected status, and risk metadata. It is designed to isolate harness compilation, not to replace open-world answer-level benchmarks such as GAIA (Mialon et al. 2024), AgentBench (Liu et al. 2024), Terminal-Bench (Merrill et al. 2026), WildToolBench (Yu et al. 2026), or MCP-Bench (Wang et al. 2025).

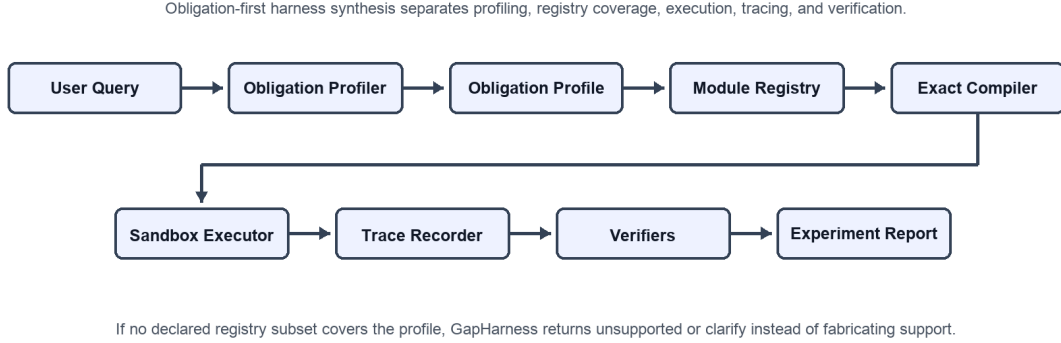


Figure 1: GapHarness pipeline. The system separates obligation profiling, declared registry lookup, exact compilation, sandbox execution, tracing, and verification.

We report success, average cost, oracle cost, minimality regret, over-harnessing, under-harnessing, wrong-harnessing, and redundancy.

4. Main Results

Table 1 shows the gold-obligation compiler result. It proves that the exact compiler matches oracle minimal harnesses under human-audited obligations. It does not prove open-world profiling.

System	N	Success	Cost	Oracle	Regret	Over	Under	Wrong
direct	1000	0.20	0.00	3.67	-3.67	0.00	0.74	0.00
tool	1000	0.34	2.10	3.67	-1.57	0.11	0.60	0.42
difficulty	1000	0.43	3.46	3.67	-0.21	0.28	0.51	0.16
always_full	1000	0.94	16.00	3.67	12.33	0.94	0.00	0.00
GH-gold	1000	1.00	3.67	3.67	0.00	0.00	0.00	0.00

5. LLM Profiler and Registry Guard

With LLM-inferred obligations on held-out test800, GapHarness reaches 0.89 success at 3.59 average cost, outperforming direct and router baselines. The main remaining failure is profiler calibration. Phase 2C adds a deterministic registry guard for a systematic sandbox/mock action failure where the LLM profiler lowered local sandbox actions into unsupported real-world side effects.

Split	Profiler	N	Success	Cost	Regret	Over	Under	Unsupported FP
test800	2B	800	0.89	3.59	-0.09	0.14	0.09	56
test800	2C	800	0.94	3.98	0.30	0.15	0.03	12
	guard							

6. Anti-Circularity Stress Tests

Registry perturbation removes key modules one at a time. For relevant 60-task subsets, base success is 1.00 and perturbed success is 0.00 for each removed module. The dominant missing capability matches the removed affordance.

Gold label permutation corrupts 200 supported profiles while the verifier checks original labels. Correct labels yield 1.00 success. Permuted labels reduce success to 0.17 and raise under-harnessing to 0.83 and wrong-harnessing to 0.79. This is an anti-circularity stress test, not a realistic label-noise model.

Stress test	Condition	N	Main result
Tool-bait negative control	GapHarness guarded	100	over-harness 0.00
Tool-bait negative control	Tool Router	100	over-harness 0.51

7. Boundaries

GAIA-Transfer gold smoke reaches 1.00 success under gold obligation labels, but this is obligation-transfer only and not GAIA answer solving. GAIA registry-guarded reaches 0.56 success with high over- and under-harnessing, a useful limitation result. Terminal-Bench-obligation50 is an appendix scaffold with labels pending audit, not Terminal-Bench solving.

The executor is deterministic sandbox/mock infrastructure. It does not perform irreversible file edits, real API calls, payments, emails, deployments, or production changes. GapBench is controlled and factorial, not a complete real-world benchmark. LLM profiling remains imperfect.

8. Conclusion

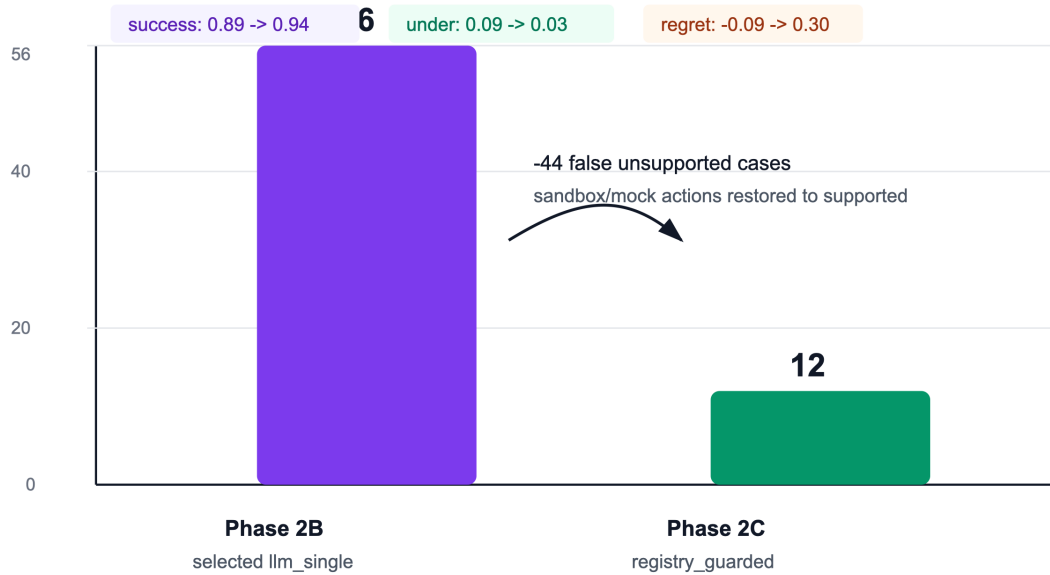
GapHarness reframes agent harnessing as two separable problems: obligation inference and minimal runtime compilation. Under gold obligations, the compiler matches oracle minimal harnesses. Under LLM-inferred obligations, it improves over simple baselines while exposing calibration tradeoffs. Registry perturbation, label permutation, and negative controls show that the system is sensitive to registry affordances and obligation semantics rather than tool keywords alone.

References

- Liu, Xiao, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, et al. 2024. “AgentBench: Evaluating LLMs as Agents.” In *International Conference on Learning Representations*. <https://openreview.net/forum?id=zAdUB0aCTQ>.
- Merrill, Mike A., Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, et al. 2026. “Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces.” *arXiv Preprint arXiv:2601.11868*. <https://arxiv.org/abs/2601.11868>.
- Mialon, Grégoire, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. “GAIA: A Benchmark for General AI Assistants.” In *International Conference on Learning Representations*. <https://arxiv.org/abs/2311.12983>.
- Patil, Shishir G., Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. “Gorilla: Large Language Model Connected with Massive APIs.” *arXiv Preprint arXiv:2305.15334*. <https://arxiv.org/abs/2305.15334>.
- Qin, Yujia, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, et al. 2024. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs.” In *International Conference on Learning Representations*. <https://openreview.net/forum?id=dHng2O0Jjr>.
- Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. “Toolformer: Language Models Can Teach Themselves to Use Tools.” In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2302.04761>.
- Wang, Zhenting, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, et al. 2025. “MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers.” *arXiv Preprint arXiv:2508.20453*. <https://arxiv.org/abs/2508.20453>.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. “ReAct: Synergizing Reasoning and Acting in Language Models.” In *International Conference on Learning Representations*. <https://arxiv.org/abs/2210.03629>.
- Yu, Peijie, Wei Liu, Yifan Yang, Jinjian Li, Zelong Zhang, Xiao Feng, and Feng Zhang. 2026. “Benchmarking LLM Tool-Use in the Wild.” *arXiv Preprint arXiv:2604.06185*. <https://arxiv.org/abs/2604.06185>.

Registry guard reduces unsupported false positives

Held-out GapBench test800, Phase 2B selected LLM vs Phase 2C registry-guarded profiler



Unsupported false positives count supported tasks compiled as unsupported. This calibration favors sufficiency and slightly raises cost/

Figure 2: Registry guard correction. Registry guarding reduces unsupported false positives from 56 to 12 on held-out GapBench test800, at the cost of higher regret.